

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 03 · LECTURE NOTES

Induction

The technique computer science uses most, and does worst.

MODULE OUTCOMES

- State and apply ordinary induction.
- Explain why the base case is not a formality.
- Use strong induction and say when it is needed.
- Use structural induction on recursive objects.
- Recognise and avoid the standard induction errors.

1 The principle

PROVE IT FOR THE FIRST CASE, AND PROVE THAT EACH CASE FORCES THE NEXT — THEN EVERY CASE FOLLOWS

Base case: the statement holds for $n = 1$, or wherever you start. **Inductive step:** if it holds for n , then it holds for $n + 1$. **Conclusion:** it holds for every n from the base onward.

And it is not circular, which is the thing to get clear early: the inductive step does not assume the statement is true — it proves an implication, 'if $P(n)$ then $P(n+1)$ ', which is a different and weaker claim that can be established without knowing whether $P(n)$ holds.

The image is a line of dominoes: you check that the first one falls, and you check that each one falling knocks over the next — **and neither of those checks assumes that they all fall.**

Which is also why both halves are genuinely required: dominoes correctly spaced but never pushed stay standing forever, and that is a base case failure rather than a technicality (§2).

CLAIM: for all $n \geq 1$, $P(n)$.

BASE CASE. $P(1)$: <show it, explicitly>

INDUCTIVE STEP. Let $n \geq 1$ and SUPPOSE $P(n)$.
 <this supposition is the inductive hypothesis;
 say so, and use the words>
 ... derivation ...
 Therefore $P(n+1)$.

By induction, $P(n)$ holds for all $n \geq 1$. []

THREE THINGS THAT MUST APPEAR

- 1 the base case, computed and not asserted
- 2 the words "suppose $P(n)$ " -- naming the inductive hypothesis explicitly
- 3 the place where you USED it

AND IF YOU CANNOT POINT AT (3), THE PROOF IS WRONG.
 This is the single most common error in the course,
 and it is detectable in about ten seconds.

Point at the place where the inductive hypothesis was used — if you cannot, the proof is wrong, and this is detectable in ten seconds by somebody who knows to look, including you. **Project 1 marks every induction proof against these three requirements**, and Project 2's self-audit is largely a matter of re-running this check on proofs you wrote six weeks earlier.

2 The base case

A VALID INDUCTIVE STEP WITH A FALSE BASE CASE PROVES A FALSE STATEMENT

'All horses are the same colour' has a perfectly valid-looking inductive step — any n horses are the same colour, so overlap two groups of n to get $n + 1$ — and it fails precisely at the transition from one horse to two, where the two groups do not overlap. Which is a base case problem in disguise: the step is valid from $n = 2$ onward and the base is at $n = 1$.

And ' $n = n + 1$ ' has a flawless inductive step: assume it, add one to both sides, and you have the next case. Only the absence of a base case stops it, and there is no base case to be had.

So check the base case by computing it, not by asserting that it is obvious — and separately check that the step actually works starting from the base, which is exactly where the horses fail and where a valid-looking argument hides a gap.

Which generalises into a discipline: the step from the base is the one most likely to be special, because it is the smallest and degenerate cases live there — and checking $n = 2$ explicitly costs nothing and catches it.

3 Strong induction

ordinary: assume $P(n)$, prove $P(n+1)$ **strong:** assume $P(1) \dots P(n)$, prove $P(n+1)$

- Ordinary induction gives you one predecessor, and that is enough whenever an object of size $n + 1$ decomposes into one of size n plus something you can handle directly.
- Strong induction gives you all the predecessors, and it is needed when the decomposition is into two pieces of unknown size — which is what every divide-and-conquer argument does: a problem of size n splits into pieces of size k and $n - k$, and you do not control k .
- And the two are equivalent in power, not in convenience — strong induction proves nothing that ordinary induction cannot (the standard trick is to induct on 'P holds for everything up to n ') — and it is frequently the only one you can actually write down.
- Strong induction is needed whenever the object splits into two pieces of unknown size, and CSCE 629 uses it constantly and will not flag it: every merge sort, quicksort, and binary search correctness argument is a strong induction, usually written without the word.
- A recursively defined object has base cases and construction rules — and you induct on the construction rather than on a natural number, which is structural induction.
- A binary tree is a leaf, or a node with two subtrees — so prove the property for a leaf, and prove it for a node while assuming it holds for both subtrees, and you are done.
- Which is exactly how every property of a recursive data structure gets proved, and is the form CSCE 629 and CSCE 605 use most — tree heights, parser correctness, and the properties of expression grammars are all this.
- And a recursive program's correctness is a structural induction on its input — the base case is the program's base case, and the inductive hypothesis is that the recursive calls work — which is why this feels like cheating the first few times and is not: you are proving an implication, as always.
- Which is why induction is the technique to be fluent in rather than merely acquainted with: it is how you reason about recursion, and recursion is everywhere in this program.

4 The standard errors

FIVE ERRORS ACCOUNT FOR NEARLY EVERY BROKEN INDUCTION PROOF

1 · No base case, or a base case asserted rather than computed (§2). **2 · The inductive hypothesis is never used** — in which case the proof is something else, possibly a correct direct proof (§1's three requirements).

3 · Proving $P(n+1)$ from scratch — which is not induction at all, and usually means you did not need induction and should say so. **4 · Assuming what you are proving** — assuming $P(n+1)$ somewhere in the derivation of $P(n+1)$, **which is circular and feels entirely fine while you are writing it.**

5 · Using strong induction's extra assumptions without having declared strong induction — reaching back to $P(k)$ for some $k < n$ in a proof that only supposed $P(n)$ — **which is the subtle one**, and is **why §3 is worth being explicit about**: the fix is one word in the hypothesis.

And all five are detectable by a single question: point at the line where the hypothesis was used, and say which predecessors it needed — which is the audit Project 2 asks you to run on your own earlier proofs, and which reliably finds something.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapter 5 (free)	The whole module , free — and the false induction proofs it works through are the best part.
MIT 6.042J, the induction lectures (free video)	§§1 to 3 , with the horses example done properly.
Hammack — Book of Proof, chapter 10 (free)	§§1 and 3 — many worked examples, free, and strong induction is handled carefully.
CLRS, the loop invariant material (library copy)	§3's application — loop invariants are induction on iterations, and this is the form CSCE 629 will use from week one.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Explain why induction is not circular**, in three sentences.
- 2 Prove the sum of the first n integers formula**, with all three requirements visible.
- 3 Find the flaw in the horses proof** precisely, and say which step fails.
- 4 Construct your own false induction** with a valid-looking step.

- 5 **Prove something needing strong induction**, and say why ordinary fails.
- 6 **Prove every integer above 1 has a prime factorisation**.
- 7 **Prove a property of binary trees** by structural induction.
- 8 **Prove a recursive function correct**, stating the hypothesis explicitly.
- 9 **Take five of your own induction proofs** and point at the hypothesis use in each.
- 10 **Find one of your proofs that uses strong induction** without declaring it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the principle and explain why it is not circular.
- 2 Give the three things that must appear in the write-up.
- 3 What is the ten-second check?
- 4 Why is a base case not a formality? Give two examples.
- 5 Where does the horses proof fail, exactly?
- 6 Contrast ordinary and strong induction, and say when each is needed.
- 7 Are they equivalent in power? In convenience?
- 8 Describe structural induction on trees.
- 9 What is the inductive hypothesis in a recursive correctness proof?
- 10 Name the five standard errors and the question that finds them all.